

PROJECT TECHNICAL DOCUMENTATION

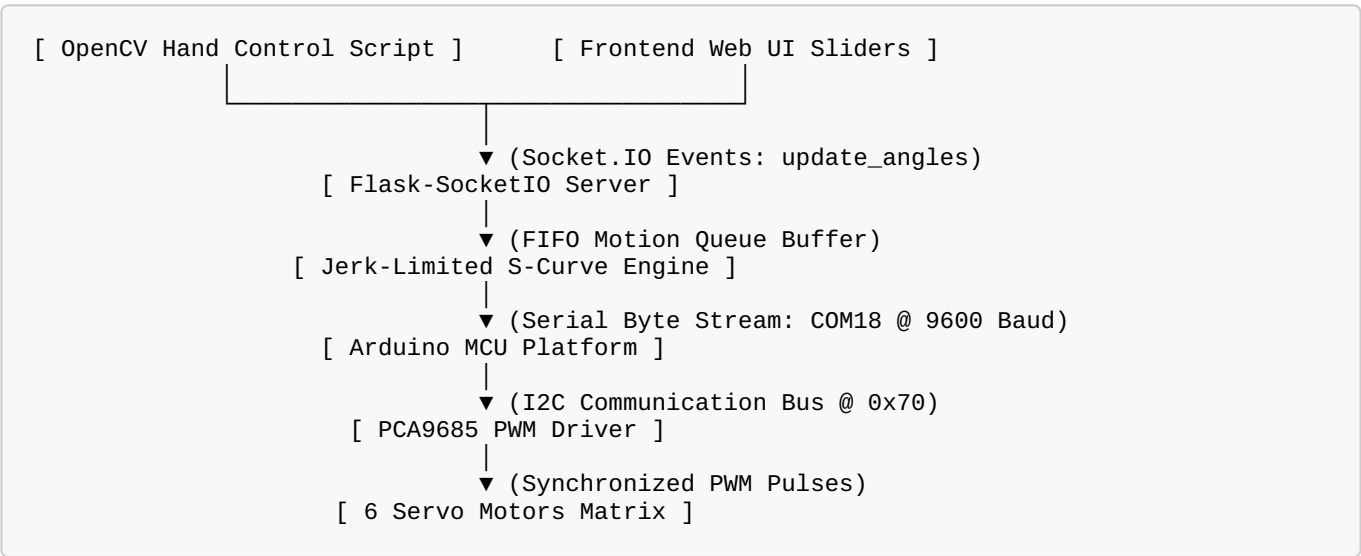
6-DOF CyberDeck Robotic Arm Controller

1. Executive Summary

This project outlines the architecture and deployment of an industrial-grade 6-DOF (Degrees of Freedom) Robotic Arm Manipulation System. The system integrates a real-time Flask-SocketIO Web Dashboard and an OpenCV + MediaPipe AI Hand Tracking Controller under a unified, multi-threaded industrial motion planning backend. Hardware abstraction and high-precision joint control are achieved using a PCA9685 I2C PWM Driver module running over an optimized data link.

2. System Architecture Layout

The system architecture employs a distributed micro-client configuration where the data transmission pipeline flows as follows:



3. Complete Source Code Registries

A. Core Application Server Backend (app.py)

Save this file as app.py. This script handles industrial S-curve acceleration filtering, client handshakes, multi-threaded task management, and communication recovery wrappers.

```

import time
import threading
import queue
import math
from typing import List, Dict, Any, Optional
import serial
from flask import Flask, render_template
from flask_socketio import SocketIO, emit

# — CONFIGURATION —
SERIAL_PORT = "COM18"
BAUD_RATE = 9600
SIMULATION_MODE = False

# — TUNABLE INDUSTRIAL MOTION PARAMETERS —
STEP_TIME = 0.01 # Time interval per interpolation step in seconds (10ms updates)
INTERPOLATION_STEPS = 50 # Number of discrete steps per individual point-to-point movement
SMOOTHNESS = 1.0 # Profile stretching modifier factor

app = Flask(__name__)
app.config["SECRET_KEY"] = "robotic-arm-advanced-key"
socketio = SocketIO(app, cors_allowed_origins="*", async_mode='threading')

ser: Optional[serial.Serial] = None
serial_lock = threading.Lock()

saved_frames: List[List[int]] = []
is_running_sequence = False
sequence_speed = 15

def init_serial(port_name: Optional[str] = None) -> bool:
    global ser, SERIAL_PORT
    if port_name:
        SERIAL_PORT = port_name

    if SIMULATION_MODE:
        print("[Serial] Simulation mode active. Hardware link bypassed.")
        return False

    with serial_lock:
        try:
            if ser is not None and ser.is_open:
                ser.close()
            ser = serial.Serial(port=SERIAL_PORT, baudrate=BAUD_RATE, timeout=1)
            time.sleep(2)
            print(f"🚀 [Serial] HARDWARE SUCCESS: Connected on physical bus {SERIAL_PORT}")
            return True
        except Exception as e:
            print(f"❌ [Serial] Connection Failed on bus target {SERIAL_PORT}: {e}")
            ser = None
            return False

def check_and_recover_serial() -> bool:
    with serial_lock:
        if ser is not None and ser.is_open:
            return True
        print(f"⚠️ [Serial] Connection drop detected. Attempting operational phase recovery...")
        return init_serial()

def send_to_serial(angles: List[float]) -> str:
    global ser
    safe_angles = [int(max(0, min(180, a))) for a in angles[:6]]
    payload = ",".join(str(a) for a in safe_angles) + "\n"
    encoded = payload.encode("utf-8")

    if SIMULATION_MODE:
        return f"[SIM MODE] {payload.strip()}"

```

```

    if ser is None or not ser.is_open:
        if not check_and_recover_serial():
            return f"[ERR] Device Unreachable offline. Simulated fallback:
{payload.strip()}"

    with serial_lock:
        try:
            ser.write(encoded)
            ser.flush()
            return f"[TX SUCCESS] {payload.strip()}"
        except Exception as e:
            print(f"✗ [Serial] Runtime write failure encountered: {e}")
            ser = None
            return f"[ERR] Link write failure: {e}"

class MotionPlanner:
    def __init__(self) -> None:
        self.current_position: List[float] = [90.0, 90.0, 90.0, 90.0, 90.0, 90.0]
        self.last_sent_position: List[int] = [-1, -1, -1, -1, -1, -1]
        self.motion_queue: queue.Queue = queue.Queue()

        self.lock = threading.Lock()
        self.state_condition = threading.Condition(self.lock)

        self.is_paused = False
        self.emergency_stop_triggered = False
        self.engine_active = False

        self.worker_thread = threading.Thread(target=self._motion_loop_processor,
daemon=True)
        self.worker_thread.start()
        print("[Planner] Synchronized Industrial Motion Core initiated.")

    def move_to(self, target_angles: List[float]) -> None:
        validated = [float(max(0, min(180, a))) for a in target_angles[:6]]
        while len(validated) < 6:
            validated.append(90.0)

        with self.lock:
            if self.emergency_stop_triggered:
                print("[Queue] Direct discard: System is locked in E-STOP state.")
                return
            self.motion_queue.put(validated)
            self.state_condition.notify_all()

    def pause(self) -> None:
        with self.lock:
            self.is_paused = True
            self.state_condition.notify_all()

    def resume(self) -> None:
        with self.lock:
            self.is_paused = False
            self.state_condition.notify_all()

    def stop(self) -> None:
        with self.lock:
            while not self.motion_queue.empty():
                try:
                    self.motion_queue.get_nowait()
                except queue.Empty:
                    break
            self.emergency_stop_triggered = True
            self.state_condition.notify_all()

    def reset_estop(self) -> None:
        with self.lock:
            self.emergency_stop_triggered = False
            self.state_condition.notify_all()

    def ease(self, t: float) -> float:
        if t <= 0.0: return 0.0
        if t >= 1.0: return 1.0
        return 0.5 * (1.0 - math.cos(t * math.pi))

```

```

    def interpolate(self, start: List[float], end: List[float], step: int, total_steps:
int) -> List[float]:
        t = step / float(total_steps)
        factor = self.ease(t)
        calculated_frame = []
        for s, e in zip(start, end):
            delta = e - s
            calculated_frame.append(s + (delta * factor))
        return calculated_frame

    def _motion_loop_processor(self) -> None:
        while True:
            with self.lock:
                while (self.motion_queue.empty() or self.is_paused or
self.emergency_stop_triggered):
                    self.engine_active = False
                    self.state_condition.wait()

                self.engine_active = True
                target_destination = self.motion_queue.get()
                start_positions = list(self.current_position)

                print(f"[Motion] Executing trajectory transition: {start_positions} ->
{target_destination}")
                steps = INTERPOLATION_STEPS
                step_idx = 1

                while step_idx <= steps:
                    with self.lock:
                        while self.is_paused and not self.emergency_stop_triggered:
                            self.state_condition.wait()

                        if self.emergency_stop_triggered:
                            break

                    computed_frame = self.interpolate(start_positions, target_destination,
step_idx, steps)
                    self.current_position = computed_frame

                    rounded_int_frame = [int(max(0, min(180, round(val)))) for val in
computed_frame]
                    if rounded_int_frame != self.last_sent_position:
                        log_msg = send_to_serial(computed_frame)
                        self.last_sent_position = rounded_int_frame
                        socketio.emit("serial_log", {"log": f"[Planner] {log_msg}"})

                    time.sleep(STEP_TIME * SMOOTHNESS)
                    step_idx += 1

                    with self.lock:
                        if not self.emergency_stop_triggered:
                            self.current_position = target_destination
                            send_to_serial(target_destination)
                            self.last_sent_position = [int(max(0, min(180, round(v)))) for v in
target_destination]
                            self.motion_queue.task_done()

motion_engine = MotionPlanner()

def sequence_runner_thread() -> None:
    global is_running_sequence, saved_frames, sequence_speed
    print("[Planner] Running automated frame sequence script tracking...")

    while is_running_sequence:
        frame_index = 0
        has_frames = False

        with serial_lock:
            if saved_frames and len(saved_frames) > 0:
                has_frames = True
                total_count = len(saved_frames)

        if not has_frames:

```

```

        is_running_sequence = False
        break

    while frame_index < total_count and is_running_sequence:
        with serial_lock:
            if frame_index >= len(saved_frames):
                break
            target_frame = saved_frames[frame_index]

            socketio.emit("serial_log", {"log": f"🔄 [AUTO] Target Frame dispatch
{frame_index + 1}: {target_frame}"})
            socketio.emit("auto_playing_frame", {"index": frame_index})

            motion_engine.move_to(target_frame)

            while True:
                with motion_engine.lock:
                    queue_empty = motion_engine.motion_queue.empty()
                    active = motion_engine.engine_active
                    stopped = motion_engine.emergency_stop_triggered

                    if stopped or not is_running_sequence:
                        break
                    if queue_empty and not active:
                        break
                    time.sleep(0.02)

            if not is_running_sequence or motion_engine.emergency_stop_triggered:
                break

            sleep_duration = 0.5 + (sequence_speed * 0.05)
            time.sleep(max(0.01, sleep_duration))
            frame_index += 1

        time.sleep(0.1)

    is_running_sequence = False
    socketio.emit("sequence_status", {"running": False})

@app.route("/")
def index() -> str:
    return render_template("index.html")

@socketio.on("connect")
def on_connect() -> None:
    mode = "simulation" if (ser is None and not SIMULATION_MODE) else "connected"
    if SIMULATION_MODE: mode = "simulation"
    emit("status", {"simulation": (ser is None), "mode": mode, "port": SERIAL_PORT,
"frames": saved_frames})

@socketio.on("update_angles")
def on_update_angles(data: Dict[str, Any]) -> None:
    global sequence_speed
    try:
        angles = data.get("data", [])
        sequence_speed = int(data.get("speed", 15))
        if isinstance(angles, list) and len(angles) >= 6:
            motion_engine.move_to(angles)
    except Exception as e:
        print(f"Error filtering incoming telemetry frame updates: {e}")

@socketio.on("change_port")
def on_change_port(data: Dict[str, Any]) -> None:
    requested_port = data.get("port", "COM18").upper().strip()
    emit("serial_log", {"log": f"⚙️ [SYS] Reconfiguring communication bus to
{requested_port}..."})
    success = init_serial(requested_port)
    mode = "simulation" if ser is None else "connected"
    emit("status", {"simulation": ser is None, "mode": mode, "port": SERIAL_PORT,
"frames": saved_frames})
    if success:
        emit("serial_log", {"log": f"✅ [SYS] Port successfully changed to
{SERIAL_PORT}!"})
    else:

```

```

        emit("serial_log", {"log": f"❌ [SYS] Failed to load {SERIAL_PORT}. Switched to
Simulation."})

@socketio.on("save_frame")
def on_save_frame(data: Dict[str, Any]) -> None:
    global saved_frames
    frame = data.get("frame", [])
    if len(frame) == 6:
        safe_frame = [int(max(0, min(180, a))) for a in frame]
        saved_frames.append(safe_frame)
        emit("update_library", {"frames": saved_frames}, broadcast=True)

@socketio.on("clear_frames")
def on_clear_frames() -> None:
    global saved_frames, is_running_sequence
    is_running_sequence = False
    motion_engine.stop()
    motion_engine.reset_estop()
    saved_frames = []
    emit("update_library", {"frames": saved_frames}, broadcast=True)

@socketio.on("run_sequence")
def on_run_sequence() -> None:
    global is_running_sequence
    if saved_frames and not is_running_sequence:
        is_running_sequence = True
        motion_engine.reset_estop()
        motion_engine.resume()
        emit("sequence_status", {"running": True})
        threading.Thread(target=sequence_runner_thread, daemon=True).start()

@socketio.on("stop_sequence")
def on_stop_sequence() -> None:
    global is_running_sequence
    is_running_sequence = False
    motion_engine.stop()
    emit("sequence_status", {"running": False})
    emit("serial_log", {"log": "🛑 [Emergency Stop] Action command halted. Buffers
purged."})
    motion_engine.reset_estop()

if __name__ == "__main__":
    init_serial()
    socketio.run(app, host="0.0.0.0", port=5000, debug=False, allow_unsafe_werkzeug=True)

```

B. AI Computer Vision Control Script (hand_control.py)

Save this file as hand_control.py. Run this script in a separate terminal while your backend server is up to initiate webcam capturing and 6-DOF coordinate gesture translation.

```

import cv2
import mediapipe as mp
import socketio
import math
import time

sio = socketio.Client()
SERVER_URL = "http://127.0.0.1:5000"

print(f"[OpenCV] Connecting to CyberDeck Server at {SERVER_URL}...")
try:
    sio.connect(SERVER_URL)
    print("[OpenCV] Connected to backend seamlessly!")
except Exception as e:
    print(f"[OpenCV] Connection failed: {e}. Make sure app.py is running first!")
    exit()

mp_hands = mp.solutions.hands
hands = mp_hands.Hands(
    static_image_mode=False,
    max_num_hands=1,
    min_detection_confidence=0.7,
    min_tracking_confidence=0.7
)
mp_draw = mp.solutions.drawing_utils
cap = cv2.VideoCapture(0)

last_send_time = 0
SEND_INTERVAL = 0.05 # 50ms rate control layer

print("\n🚀 Full 6-DOF Hand Tracking Active. Show your hand to the webcam...")
while cap.isOpened():
    success, frame = cap.read()
    if not success: continue

    frame = cv2.flip(frame, 1)
    h, w, c = frame.shape
    rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    results = hands.process(rgb_frame)

    if results.multi_hand_landmarks:
        for hand_landmarks in results.multi_hand_landmarks:
            mp_draw.draw_landmarks(frame, hand_landmarks, mp_hands.HAND_CONNECTIONS)

            # --- LANDMARK VECTOR EXTRACTION ---
            wrist = hand_landmarks.landmark[0]
            thumb_tip = hand_landmarks.landmark[4]
            index_mcp = hand_landmarks.landmark[5]
            index_tip = hand_landmarks.landmark[8]
            middle_tip = hand_landmarks.landmark[12]
            pinky_mcp = hand_landmarks.landmark[17]
            pinky_tip = hand_landmarks.landmark[20]

            # 1. Base Rotation (J1): X-axis wrist parsing bounds
            j1_base = int((wrist.x - 0.2) * (180 / 0.6))
            j1_base = max(0, min(180, j1_base))

            # 2. Shoulder Pitch (J2): Y-axis wrist parsing bounds
            j2_should = int((wrist.y - 0.3) * (180 / 0.5))
            j2_should = max(0, min(180, j2_should))

            # 3. Elbow Pitch (J3): Index finger tip scalar depth tracking
            j3_elbow = int((index_tip.y - 0.2) * (180 / 0.6))
            j3_elbow = max(0, min(180, j3_elbow))

            # 4. Wrist Pitch (J4): Geometric linear mapping from middle finger to wrist
            pitch_dist = math.sqrt((middle_tip.x - wrist.x)**2 + (middle_tip.y -
wrist.y)**2)
            j4_wrist_pitch = int((pitch_dist - 0.25) * (180 / 0.30))
            j4_wrist_pitch = max(0, min(180, j4_wrist_pitch))

            # 5. Wrist Roll (J5): Rotational slope tracking via knuckles vector
            angle_radians = math.atan2(pinky_mcp.y - index_mcp.y, pinky_mcp.x -

```

```

index_mcp.x)
    angle_degrees = math.degrees(angle_radians)
    j5_wrist_roll = int((angle_degrees + 45) * (180 / 90))
    j5_wrist_roll = max(0, min(180, j5_wrist_roll))

    # 6. Gripper Claw (J6): Spatial distance calculation between thumb and pinky
tips
    gripper_dist = math.sqrt((thumb_tip.x - pinky_tip.x)**2 + (thumb_tip.y -
pinky_tip.y)**2)
    j6_gripper = int((gripper_dist - 0.05) * (180 / 0.20))
    j6_gripper = max(0, min(180, j6_gripper))

    current_angles = [j1_base, j2_shoulder, j3_elbow, j4_wrist_pitch,
j5_wrist_roll, j6_gripper]

    current_time = time.time()
    if current_time - last_send_time > SEND_INTERVAL:
        sio.emit("update_angles", {"data": current_angles, "speed": 10})
        last_send_time = current_time

    cv2.putText(frame, f"6-DOF Matrix: {current_angles}", (10, 30),
cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 170), 2)

    cv2.imshow("6-DOF CyberDeck OpenCV Matrix Engine", frame)
    if cv2.waitKey(1) & 0xFF == ord('q'): break

cap.release()
cv2.destroyAllWindows()
sio.disconnect()

```

C. Frontend Dashboard Component (templates/index.html)

Save this file as index.html inside a subfolder named templates (i.e., templates/index.html).

```

<!DOCTYPE html><html lang="en"><head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>6-DOF Matrix CyberDeck</title>
  <script src="https://cdnjs.cloudflare.com/ajax/libs/socket.io/4.0.1/socket.io.js"></
script>
  <style>
    :root {
      --bg-color: #040712; --panel-bg: rgba(13, 18, 35, 0.75);
      --panel-border: rgba(0, 210, 255, 0.2); --accent-blue: #00d2ff;
      --accent-glow: #0066ff; --accent-green: #00ffaa; --accent-red: #ff0055;
      --text-main: #e2e8f0; --text-muted: #4b5e7d; --slider-track: #0f172a;
      --slider-border: rgba(0, 210, 255, 0.4); --canvas-opacity: 0.18;
    }
    [data-theme="light"] {
      --bg-color: #f1f5f9; --panel-bg: rgba(255, 255, 255, 0.85);
      --panel-border: rgba(59, 130, 246, 0.3); --accent-blue: #2563eb;
      --accent-glow: #3b82f6; --accent-green: #10b981; --accent-red: #ef4444;
      --text-main: #1e293b; --text-muted: #64748b; --slider-track: #e2e8f0;
      --slider-border: rgba(37, 99, 235, 0.4); --canvas-opacity: 0.08;
    }
    body {
      margin: 0; font-family: 'Segoe UI', Roboto, sans-serif;
      background-color: var(--bg-color); color: var(--text-main);
      min-height: 100vh; display: flex; flex-direction: column; overflow: hidden;
      transition: background-color 0.4s ease, color 0.3s ease;
    }
    #math-canvas {
      position: absolute; top: 0; left: 0; width: 100vw; height: 100vh;
      z-index: -1; opacity: var(--canvas-opacity); pointer-events: none; transition:
opacity 0.4s ease;
    }
    header {
      background: rgba(10, 16, 32, 0.7); border-bottom: 1px solid var(--panel-
border);
      backdrop-filter: blur(15px); padding: 12px 25px; display: flex;
      justify-content: space-between; align-items: center; z-index: 10;
    }
    [data-theme="light"] header { background: rgba(255, 255, 255, 0.7); }
    .logo h1 { margin: 0; font-size: 18px; letter-spacing: 2px; text-transform:
uppercase; }
    .system-status-bar { display: flex; gap: 15px; align-items: center; }
    .stat-widget {
      background: rgba(255,255,255,0.04); border: 1px solid var(--panel-border);
      padding: 5px 12px; border-radius: 6px; display: flex; align-items: center;
gap: 8px; font-size: 12px;
    }
    [data-theme="light"] .stat-widget { background: rgba(0,0,0,0.02); }
    .port-input-wrapper { display: flex; align-items: center; gap: 4px; }
    .port-input {
      background: rgba(0,0,0,0.3); border: 1px solid var(--panel-border); color:
var(--accent-blue);
      font-family: monospace; font-weight: bold; padding: 2px 6px; border-radius:
4px; width: 70px; text-align: center; outline: none;
    }
    [data-theme="light"] .port-input { background: #fff; }
    .port-btn { background: var(--accent-blue); color: #000; border: none; padding:
3px 8px; font-size: 10px; border-radius: 4px; cursor: pointer; font-weight: bold; }
    .status-dot { width: 8px; height: 8px; background: var(--accent-red); border-
radius: 50%; }
    .status-dot.connected { background: var(--accent-green); box-shadow: 0 0 8px
var(--accent-green); }
    .dashboard-container { display: grid; grid-template-columns: 380px 1fr 340px; gap:
20px; padding: 20px; flex-grow: 1; height: calc(100vh - 85px); box-sizing: border-box; z-
index: 2; }
    .panel { background: var(--panel-bg); border: 1px solid var(--panel-border);
border-radius: 14px; backdrop-filter: blur(12px); padding: 20px; display: flex; flex-
direction: column; box-shadow: 0 8px 32px rgba(0,0,0,0.25); overflow: hidden; transition:
background 0.4s ease, border 0.4s ease; }
    .panel-header { font-size: 12px; text-transform: uppercase; letter-spacing: 2px;
color: var(--accent-blue); margin-bottom: 15px; border-bottom: 1px solid var(--panel-
border); padding-bottom: 8px; font-weight: bold; }
    .controls-scroll { overflow-y: auto; flex-grow: 1; }

```

```

.controls-scroll::-webkit-scrollbar { width: 3px; }
.joint-wrapper { background: rgba(255,255,255,0.01); border: 1px solid
rgba(255,255,255,0.03); border-radius: 8px; padding: 12px; margin-bottom: 12px; }
[data-theme="light"] .joint-wrapper { background: rgba(0,0,0,0.01); }
.joint-meta { display: flex; justify-content: space-between; font-size: 12px;
margin-bottom: 8px; }
.joint-deg { color: var(--accent-blue); font-weight: bold; }
input[type=range] { width: 100%; -webkit-appearance: none; background: var(--text-
muted); height: 6px; border-radius: 10px; outline: none; border: 1px solid var(--slider-
border); box-shadow: inset 0 1px 3px rgba(0,0,0,0.5); }
input[type=range]::-webkit-slider-thumb { -webkit-appearance: none; width: 16px;
height: 16px; border-radius: 50%; background: var(--accent-blue); box-shadow: 0 0 12px
var(--accent-glow); cursor: pointer; }
.center-column { display: grid; grid-template-rows: 1fr 170px; gap: 20px; }
.canvas-workspace { display: flex; flex-direction: column; justify-content:
center; align-items: center; }
.vector-text { font-size: 38px; font-weight: 800; font-family: monospace; text-
shadow: 0 0 15px rgba(0,210,255,0.3); }
.timeline-container { width: 80%; background: var(--slider-track); height: 6px;
border-radius: 10px; margin-top: 20px; overflow: hidden; display: none; border: 1px solid
var(--panel-border); }
.timeline-fill { height: 100%; width: 0%; background: linear-gradient(90deg,
var(--accent-blue), var(--accent-green)); }
.library-scroller { overflow-y: auto; flex-grow: 1; margin: 10px 0; }
.frame-node { background: rgba(255,255,255,0.02); border: 1px solid var(--panel-
border); padding: 10px; border-radius: 8px; margin-bottom: 8px; font-size: 11px; font-
family: monospace; display: flex; justify-content: space-between; align-items: center;
border-left: 4px solid var(--text-muted); }
[data-theme="light"] .frame-node { background: rgba(0,0,0,0.02); }
.frame-node.active-play { border-left-color: var(--accent-green); background:
rgba(5, 255, 176, 0.1); }
button { font-family: inherit; background: rgba(255,255,255,0.05); color: var(--
text-main); border: 1px solid var(--panel-border); padding: 10px 14px; border-radius: 8px;
cursor: pointer; font-size: 11px; font-weight: 600; letter-spacing: 1px; transition: all
0.2s; }
[data-theme="light"] button { background: rgba(0,0,0,0.05); }
button:hover { background: var(--accent-blue); color: #000; box-shadow: 0 0 10px
var(--accent-glow); }
button.success-btn { color: var(--accent-green); border-color: var(--accent-
green); }
button.success-btn:hover { background: var(--accent-green); color: #000; }
button.danger-btn { color: var(--accent-red); border-color: var(--accent-red); }
button.danger-btn:hover { background: var(--accent-red); color: #fff; }
.estop-btn { background: linear-gradient(135deg, #7f1d1d 0%, #b91c1c 100%);
border: 2px solid var(--accent-red); color: #fff; font-weight: bold; width: 100%; padding:
12px; border-radius: 8px; margin-top: 10px; }
.estop-btn:hover { background: var(--accent-red); color: #fff; box-shadow: 0 0 20px
var(--accent-red); }
.monitor-log { background: rgba(0,0,0,0.4); border: 1px solid var(--panel-border);
border-radius: 8px; padding: 12px; font-family: monospace; font-size: 11px; color: var(--
accent-green); overflow-y: auto; flex-grow: 1; }
[data-theme="light"] .monitor-log { background: #000; }
</style></head><body>
<canvas id="math-canvas"></canvas>
<header>
<div class="logo"><h1>CyberDeck Systems</h1></div>
<div class="system-status-bar">
<div class="stat-widget"><span style="color:var(--text-muted)">Port Bus:</
span>
<div class="port-input-wrapper">
<input type="text" class="port-input" id="comPortInput" value="COM18">
<button class="port-btn" onclick="applyNewPort()">Apply</button>
</div>
</div>
<div class="stat-widget"><span class="status-dot" id="status-indicator"></
span><span id="mode-text" style="font-weight:bold;">OFFLINE</span></div>
<button onclick="toggleTheme()" style="padding: 5px 12px; font-size:11px;">🌑
Theme</button>
</div>
</header>
<div class="dashboard-container">
<div class="panel">
<div class="panel-header">Joint Servo Matrix</div>
<div class="controls-scroll">

```

```

        <div class="joint-wrapper">
            <div class="joint-meta"><span>J1 - Base Rotation</span><span
class="joint-deg" id="v1">90°</span></div>
            <input type="range" min="0" max="180" value="90" id="j1"
oninput="updateAngles()">
        </div>
        <div class="joint-wrapper">
            <div class="joint-meta"><span>J2 - Shoulder Pitch</span><span
class="joint-deg" id="v2">90°</span></div>
            <input type="range" min="0" max="180" value="90" id="j2"
oninput="updateAngles()">
        </div>
        <div class="joint-wrapper">
            <div class="joint-meta"><span>J3 - Elbow Pitch</span><span
class="joint-deg" id="v3">90°</span></div>
            <input type="range" min="0" max="180" value="90" id="j3"
oninput="updateAngles()">
        </div>
        <div class="joint-wrapper">
            <div class="joint-meta"><span>J4 - Wrist Pitch</span><span
class="joint-deg" id="v4">90°</span></div>
            <input type="range" min="0" max="180" value="90" id="j4"
oninput="updateAngles()">
        </div>
        <div class="joint-wrapper">
            <div class="joint-meta"><span>J5 - Wrist Roll</span><span
class="joint-deg" id="v5">90°</span></div>
            <input type="range" min="0" max="180" value="90" id="j5"
oninput="updateAngles()">
        </div>
        <div class="joint-wrapper">
            <div class="joint-meta"><span>J6 - Gripper Claw</span><span
class="joint-deg" id="v6">90°</span></div>
            <input type="range" min="0" max="180" value="90" id="j6"
oninput="updateAngles()">
        </div>
        <div class="joint-wrapper" style="border-color: var(--panel-border);">
            <div class="joint-meta"><span style="color:#f59e0b;">Sequence Delay
Factor</span><span id="vSpeed" style="color:#f59e0b;">15</span></div>
            <input type="range" min="1" max="50" value="15" id="speedSlider"
oninput="updateAngles()">
        </div>
        <button onclick="homePosition()" style="margin-top:10px;">Calibrate Home</
button>
    </div>
    <div class="center-column">
        <div class="panel canvas-workspace">
            <div class="vector-text" id="array-display">[90, 90, 90, 90, 90, 90]</div>
            <span style="font-size:11px; color:var(--text-muted); text-
transform:uppercase; letter-spacing:1px; margin-top:5px;">Kinematics Telemetry Signal</
span>
            <div class="timeline-container" id="playback-timeline"><div
class="timeline-fill" id="timeline-bar"></div></div>
            <div class="panel">
                <div class="panel-header">Data Stream Bus Log</div>
                <div class="monitor-log" id="log-screen"></div>
            </div>
        </div>
        <div class="panel">
            <div class="panel-header">Vector Automation</div>
            <button class="success-btn" onclick="saveFrame()">📌 Index Current Vector</
button>
            <div style="display:flex; justify-content:space-between; font-size:11px;
margin:15px 0 5px 0; color:var(--text-muted);">
                <span>Stored Registries</span><span id="frame-counter" style="color:var(--
accent-blue)">0 Frames</span>
            </div>
            <div class="library-scroller" id="library-box"></div>
            <div style="display:grid; grid-template-columns: 1fr 1fr; gap:10px; margin-
top:auto;">
                <button id="runBtn" class="success-btn" onclick="toggleSequence()">▶ Run
Loop</button>

```

```

        <button class="danger-btn" onclick="clearFrames()">🗑️ Wipe All</button>
    </div>
    <button class="estop-btn" onclick="emergencyStop()">⚠️ EMERGENCY STOP (E-
STOP)</button>
</div>
</div>
<script>
    const socket = io("http://" + window.location.hostname + ":" +
window.location.port);
    const logScreen = document.getElementById("log-screen");
    const statusIndicator = document.getElementById("status-indicator");
    const modeText = document.getElementById("mode-text");
    let isRunning = false; let totalFramesCount = 0;

    const canvas = document.getElementById('math-canvas'); const ctx =
canvas.getContext('2d');
    let symbols = ['Σ', 'f', 'π', '√x', 'Δ', '∞', 'λ', 'Ω', 'θ', 'f(x)']; let fontArr
= [];
    function initCanvas() {
        canvas.width = window.innerWidth; canvas.height = window.innerHeight; fontArr
= [];
        let columns = Math.floor(canvas.width / 40);
        for(let i=0; i<columns; i++) {
            fontArr.push({ x: i * 40, y: Math.random() * canvas.height, speed: 0.5 +
Math.random() * 1.2, char: symbols[Math.floor(Math.random() * symbols.length)], size: 10 +
Math.floor(Math.random() * 16) });
        }
    }
    function animateCanvas() {
        ctx.clearRect(0, 0, canvas.width, canvas.height);
        ctx.fillStyle = document.documentElement.getAttribute('data-theme') ===
'light' ? '#2563eb' : '#00d2ff';
        fontArr.forEach(p => {
            ctx.font = `${p.size}px Courier New`; ctx.fillText(p.char, p.x, p.y); p.y
+= p.speed;
            if(p.y > canvas.height) { p.y = -20; p.char =
symbols[Math.floor(Math.random() * symbols.length)]; }
        });
        requestAnimationFrame(animateCanvas);
    }
    window.addEventListener('resize', initCanvas); initCanvas(); animateCanvas();

    socket.on("connect", () => { statusIndicator.className = "status-dot
connected"; });
    socket.on("status", (status) => {
        document.getElementById("comPortInput").value = status.port;
        modeText.innerText = !status.simulation ? "HARDWARE LINKED" : "SIMULATION
ACTIVE";
        modeText.style.color = !status.simulation ? "var(--accent-green)" : "#ff9900";
        renderLibrary(status.frames || []);
    });
    socket.on("serial_log", (msg) => { logScreen.innerHTML += `${msg.log}<br>`;
logScreen.scrollTop = logScreen.scrollHeight; });
    socket.on("update_library", (data) => { renderLibrary(data.frames); });
    socket.on("sequence_status", (status) => {
        isRunning = status.running; const btn = document.getElementById("runBtn");
        btn.innerText = isRunning ? "🛑 Stop Loop" : "▶️ Run Loop";
        btn.className = isRunning ? "danger-btn" : "success-btn";
    });
    socket.on("auto_playing_frame", (data) => {
        document.querySelectorAll('.frame-node').forEach((el, idx) => {
            if(idx === data.index) el.classList.add('active-play'); else
el.classList.remove('active-play');
        });
    });
    function toggleTheme() {
        let current = document.documentElement.getAttribute('data-theme');
        document.documentElement.setAttribute('data-theme', current === 'light' ?
'dark' : 'light');
    }
    function applyNewPort() { socket.emit("change_port", { port:
document.getElementById("comPortInput").value }); }
    function getCurrentAngles() {
        return [parseInt(document.getElementById("j1").value),

```

```

parseInt(document.getElementById("j2").value),
parseInt(document.getElementById("j3").value),
parseInt(document.getElementById("j4").value),
parseInt(document.getElementById("j5").value),
parseInt(document.getElementById("j6").value)];
}
function updateAngles() {
    let angles = getCurrentAngles(); let speed =
document.getElementById("speedSlider").value;
    document.getElementById("vSpeed").innerText = speed;
    for(let i=1; i<=6; i++) document.getElementById(`v${i}`).innerText =
angles[i-1] + "°";
    document.getElementById("array-display").innerText = `[${angles.join(', ')}]`;
    socket.emit("update_angles", { data: angles, speed: speed });
}
function saveFrame() { socket.emit("save_frame", { frame: getCurrentAngles() }); }
function clearFrames() { socket.emit("clear_frames"); }
function toggleSequence() { if(isRunning) socket.emit("stop_sequence"); else
socket.emit("run_sequence"); }
function emergencyStop() { socket.emit("stop_sequence"); homePosition(); }
function renderLibrary(frames) {
    const box = document.getElementById("library-box"); totalFramesCount =
frames.length;
    document.getElementById("frame-counter").innerText = `${totalFramesCount}
Indexed`;
    if(frames.length === 0) { box.innerHTML = `<div style="color:var(--text-
muted); font-size:11px; text-align:center; padding-top:40px;">No positions indexed yet.</
div>`; return; }
    box.innerHTML = "";
    frames.forEach((f, i) => { box.innerHTML += `<div class="frame-node" id="f-$
{i}"><span><strong>#0${i+1}</strong> Node</span><span style="color:var(--accent-blue)">[$
{f.join(', ')}]</span></div>`; });
}
function homePosition() { for(let i=1; i<=6; i++) document.getElementById(`j${i}
`).value = 90; updateAngles(); }
</script></body></html>

```

D. PCA9685 Embedded Driver Microcode (firmware.ino)

Upload this firmware sketch directly to your Arduino MCU via the Arduino IDE using the default address 0x40.

```

#include <Wire.h>
#include <Adafruit_PWMServoDriver.h>

Adafruit_PWMServoDriver pwm = Adafruit_PWMServoDriver(); // Default address 0x40 mapping

#define SERVOMIN 150 // 0 degrees pulse representation
#define SERVOMAX 600 // 180 degrees pulse representation

int currentAngles[6] = {90, 90, 90, 90, 90, 90};
String inputBuffer = "";

void setup() {
  Serial.begin(9600);
  inputBuffer.reserve(64);

  pwm.begin();
  pwm.setPWMFreq(50); // Analog Servos run optimally on a 50Hz hardware frequency window

  // Bring all joints smoothly to home state positions
  for (int i = 0; i < 6; i++) {
    pwm.setPWM(i, 0, angleToPulse(currentAngles[i]));
  }
  Serial.println("[HW ready] 6-DOF PCA9685 Module Node Online.");
}

void loop() {
  while (Serial.available() > 0) {
    char inChar = (char)Serial.read();
    if (inChar == '\n') {
      processIncomingData(inputBuffer);
      inputBuffer = "";
    } else {
      inputBuffer += inChar;
    }
  }
}

int angleToPulse(int angle) {
  return map(angle, 0, 180, SERVOMIN, SERVOMAX);
}

void processIncomingData(String data) {
  data.trim();
  if (data.length() == 0) return;

  int parsedAngles[6];
  int elementIndex = 0;
  int lastDelimiter = 0;

  for (int i = 0; i < data.length(); i++) {
    if (data.charAt(i) == ',') {
      if (elementIndex < 5) {
        parsedAngles[elementIndex] = data.substring(lastDelimiter, i).toInt();
        lastDelimiter = i + 1;
        elementIndex++;
      }
    }
  }

  if (elementIndex == 5) {
    parsedAngles[5] = data.substring(lastDelimiter).toInt();

    for (int j = 0; j < 6; j++) {
      int safeAngle = constrain(parsedAngles[j], 0, 180);

      // Hardware writes occur exclusively upon delta changes to prevent bus saturation
      if (safeAngle != currentAngles[j]) {
        pwm.setPWM(j, 0, angleToPulse(safeAngle));
        currentAngles[j] = safeAngle;
      }
    }
  }
}

```

```
}  
}
```

4. Hardware Wiring Interface Specification

I. Logic Bus Setup (I2C Connections)

PCA9685 Pin	Connection Details
SDA Pin	Arduino UNO/Nano A4 (Serial Data Line)
SCL Pin	Arduino UNO/Nano A5 (Serial Clock Line)
VCC Pin	Arduino 5V / 3.3V Pin (Logic Gate Power)
GND Pin	Arduino GND Pin (System Comm Reference Ground)

II. External High-Current Power Loop

- **PCA9685 Blue/Green Terminal Screw Block V+** → External Power Adapter/Battery Positive (+) Terminal (5V - 6V DC, ≥ 3A - 5A).
- **PCA9685 Blue/Green Terminal Screw Block GND** → External Power Adapter/Battery Negative (-) Terminal.

Grounding Constraint: The external power source's GND and the Arduino's GND must be physically connected to maintain a common signal reference across the system components.

III. Driver Output Channel Mapping

Channel	Mechanism / Servo Function
Channel 0	Base Servo Rotation (J1)
Channel 1	Shoulder Pitch Servo (J2)
Channel 2	Elbow Pitch Servo (J3)
Channel 3	Wrist Pitch Servo (J4)
Channel 4	Wrist Roll Servo (J5)
Channel 5	Gripper Claw Mechanism (J6 - 6th Axis Motor)

5. Production Execution Blueprint

To boot up the complete pipeline without data flow collisions, open separate terminal shell prompts and follow this execution layout exactly:

Step 1: Install Dependencies

```
pip install flask flask-socketio pyserial opencv-python mediapipe python-socketio
```

Step 2: Spin Up Core Application Web Server (Terminal 1)

```
python app.py
```

Verify console logs display a hardware confirmation state:

 [Serial] HARDWARE SUCCESS: Connected on physical bus COM18

Step 3: Run Computer Vision Engine (Terminal 2)

```
python hand_control.py
```

Step 4: Access Web Management Console

Launch any modern web browser interface and browse to:

<http://127.0.0.1:5000>